

1) Context Window Consistency Check (CWC) — verificação de coerência com o contexto

Objetivo técnico

Detectar respostas que **contradizem** ou **ignoram evidências** presentes no contexto imediato (documentos recuperados, histórico de conversa, tool results). O foco é reduzir **alucinações de “desancoragem”** (output não sustentado pelo que o modelo “viu” na janela).

Escopo (MVP)

- Cobertura de **PT/EN**.
 - Contexto até ~20 trechos (sentenças ou passagens curtas) relevantes.
 - Combinação **barata e rápida**: *retrieval* + *NLI leve* + *checagem factual simples*.
 - Latência adicional alvo p95: $\leq 60\text{--}90\text{ ms}$ (em CPU/serviço leve; GPU opcional).
-

Entradas e saídas

Entradas

- `input`: prompt do usuário (string).
- `context_items`: lista ordenada de passagens relevantes (texto curto + metadados).
- `llm_output`: resposta textual candidata da LLM.

Saídas

- `delta_score` $\in [0, 1]$: coerência com contexto (1 = sem contradições, com suporte).
 - `flags`: lista de flags ("contradiction", "unsupported", "numerical_mismatch").
 - `evidence`: mapeamento claim $\rightarrow$ sentenças de suporte/contra-exemplo.
 - `decision_hint`: "allow" | "review" | "block" (sinal para o orquestrador do MVP).
-

Arquitetura no KAI Core (MVP)

1. **Claim Extractor (CE)**: extrai **afirmações verificáveis** da resposta (fatos, números, entidades + relações).

- Heurística: split por sentenças + regex para números/datas + NER (spacy) + filtros.

2. **Retriever Local (RL):** para cada claim, recupera **top-k** sentenças mais relevantes do `context_items`.
 - BM25/Elastic ou embeddings (e5-small / MiniLM) + *reranking* leve (cross-encoder mini).
 3. **NLI Verifier (NV):** roda **NLI binário** (entailment vs. contradiction) entre claim e cada sentença candidata.
 - Modelo leve (ex.: DeBERTa-v3-base-MNLI distilado) ou API barata.
 4. **Numeric/Temporal Checker (NTC):** verificações de **números e datas** (ex.: “10,75%” vs “11%”; “ontem/hoje/2023”).
 - Heurísticas determinísticas + tolerância (ex.: $\pm 1-2\%$).
 5. **Score Aggregator (SA):** agrega sinais em `delta_score` e gera `flags` + `evidence`.
- Obs.: no MVP, tudo é **stateless** (por request). Caching opcional de embeddings.
-

Lógica de decisão (MVP)

- Se existir **qualquer contradição forte** (NLI=contradiction com alta confiança) → `flags += contradiction`.
 - Se a maioria dos claims **não encontrar suporte** ($\text{score} < \tau$) → `flags += unsupported`.
 - Se **números/datas divergem** além do limiar → `flags += numerical_mismatch`.
 - Heurística final:
 - $\text{delta} \geq 0.85$ → `decision_hint = "allow"`.
 - $0.70 \leq \text{delta} < 0.85$ → `"review"`.
 - < 0.70 ou `flags` severos → `"block"` (ou “review com bloqueio provisório”, conforme política do piloto).
-

Pseudocódigo (realista, pronto para engenharia)

```
def context_window_consistency_check(input_text, llm_output, context_items,
    cfg):
    # 1) Extrair claims verificáveis
    claims = extract_claims(llm_output, cfg.claims)
    if not claims:
        return score(0.9), [], {}, "review" # Sem claims: tende a seguro, mas
    reveja

    flags = []
    evidence = {}
    per_claim_scores = []
```

```

# 2) Pré-index: flatenar context em sentenças curtas
ctx_sents = sentence_split(context_items) # [(sent, meta), ...]
index = build_light_index(ctx_sents, method=cfg.retriever) # BM25 ou
embeddings

for claim in claims:
    # 3) Recuperar top-k sentenças candidatas
    cands = index.retrieve(claim.text, top_k=cfg.top_k)

    # 4) Reranking leve (opcional)
    cands = rerank_cross_encoder(claim.text, cands, model=cfg.reranker,
top_m=cfg.top_m)

    # 5) NLI: entailment vs contradiction
    nli_results = [nli_judge(claim.text, s.sent) for s in cands]
    entail_scores = [r.entail_prob for r in nli_results]
    contrad_scores = [r.contrad_prob for r in nli_results]

    # 6) Num/Temporal check
    num_temporal = numeric_temporal_check(claim, cands, cfg.numeric)

    # 7) Agregação por claim
    support = max(entail_scores) if entail_scores else 0.0
    conflict = max(contrad_scores) if contrad_scores else 0.0

    claim_score = (cfg.w_entail * support) - (cfg.w_contrad * conflict)
    claim_score = clamp(0.0, 1.0, (claim_score + 1.0) / 2.0) # normaliza p/
[0,1]

    # Penalizar divergência num/temporal
    if num_temporal.mismatch:
        claim_score *= cfg.numeric_penalty
        flags.append("numerical_mismatch")

    # Flags específicas
    if conflict >= cfg.contrad_hard:
        flags.append("contradiction")

    if support < cfg.support_floor:
        flags.append("unsupported")

    per_claim_scores.append(claim_score)
    evidence[claim.text] = {
        "best_support": best_support_sentence(cands, entail_scores),
        "strongest_conflict": best_conflict_sentence(cands, contrad_scores),
        "numeric_check": num_temporal.report
    }

# 8) Score final (média ponderada por importância do claim)
delta = weighted_mean(per_claim_scores, weights=claim_importance(claims))

# 9) Decisão sugerida
if ("contradiction" in flags and delta < cfg.block_delta) or delta <
cfg.block_floor:
    hint = "block"
elif delta < cfg.review_delta or "numerical_mismatch" in flags:
    hint = "review"
else:
    hint = "allow"

return delta, dedup(flags), evidence, hint

```

Configurações sugeridas (MVP):

- `top_k=12, top_m=6; w_entail=0.8, w_contrad=1.0.`
 - `support_floor=0.55, contrad_hard=0.70.`
 - Penalização numérica: `numeric_penalty=0.85.`
 - Limiar decisão: `block_floor=0.55, review_delta=0.80, block_delta=0.65.`
-

Indicadores, testes e riscos

KPIs técnicos

- **Precision/Recall de contradição** (em conjunto rotulado): alvo **P > 0.80, R > 0.75.**
- **Precisão de detecção numérica/temporal:** > 90% em casos claros.
- **Latência adicional p95:** ≤ 90 ms (CPU); ≤ 50 ms (se reranker/NLI em GPU leve).
- **Taxa de falsos positivos** (respostas válidas marcadas): < 10% em domínio-alvo.

Testes mínimos

- **Conjunto sintético:** 300–500 pares (claim, contexto) com rótulo de (suportado / contradito / não suportado).
- **Casos adversariais:** negações indiretas, ambiguidade temporal, valores decimais no PT (vírgula) vs EN (ponto).
- **Ablation study:** NLI-only vs NLI+numeric vs NLI+numeric+rerank.

Riscos e mitigação

- **NLI frágil a língua/variação** → usar modelo multilíngue ou distilar para PT/EN; fallback heurístico de negação.
 - **Contexto ruidoso** → *reranking* e filtros para priorizar passagens com entidades/nums coincidentes.
 - **Custo/latência** → ativar CWC no **deep path** apenas quando o “risk prefilter” sinalizar incerteza; usar cache de embeddings.
 - **Cobertura limitada de “fato externo”** (fora do contexto) → esse mecanismo não “busca na web”; o alvo do CWC é **consistência com o que já está no contexto**. Factualidade externa é tratada no mecanismo #2 (Grounding & Evidence Match).
-

Integração no MVP (Base44)

- **Onde encaixa:** KAI Core → pipeline “Deep Path”.
- **Dependências mínimas:**

- Lib de NER/sentenças (spaCy), retriever (Elasticsearch/BM25 ou FAISS + MiniLM), NLI leve (HF Inference / modelo distilado).
 - **Interfaces (JSON):**
 - Entrada: `{input, llm_output, context_items:[{text, meta}...]}`
 - Saída: `{delta_score, flags:[...], evidence:{...}, decision_hint}`.
 - **Observabilidade:** logar `claim` → `evidência` (para auditoria) e métricas de taxa de conflito/suporte.
-

Escalonamento futuro (além do MVP)

- Substituir NLI leve por **cross-encoder treinado no seu domínio** (ganho de precisão).
 - Adicionar **verificação tabular** (ex.: comparar números em tabelas PDF/CSV com o `claim`).
 - Normalização numérica robusta (moedas, % com vírgula/ponto, unidades).
 - *Caching* semântico por **intent** para reaproveitar verificações de prompts similares.
-

Resultado esperado no MVP: queda visível nas alucinações que contradizem o próprio contexto e melhora de confiança dos testers, **sem** depender de retreinamento da LLM. Pronto para ser plugado no gateway/middleware do protótipo.

2) Grounding & Evidence Match — Especificação MVP

1) Objetivo Técnico

Reduzir alucinações e afirmações não-suportadas pelo modelo, exigindo **evidência verificável** antes de liberar a resposta. O módulo:

- extrai *claims* (afirmações verificáveis) do output da LLM;
- procura evidências no **contexto fornecido** (documentos do cliente, snippets de RAG) e em **fontes internas autorizadas**;
- mede aderência resposta ↔ evidência;
- anota o texto com referências (citations) e **bloqueia** / **reescreve** trechos sem suporte.

Ganhos esperados (MVP):

- queda de 30–50% em alucinações factuais em tarefas com contexto fornecido (Q&A, suporte, FAQ, relatórios);
 - trilha de auditoria e métricas por claim (quem/onde suportou).
-

2) Entradas e Saídas

Entradas

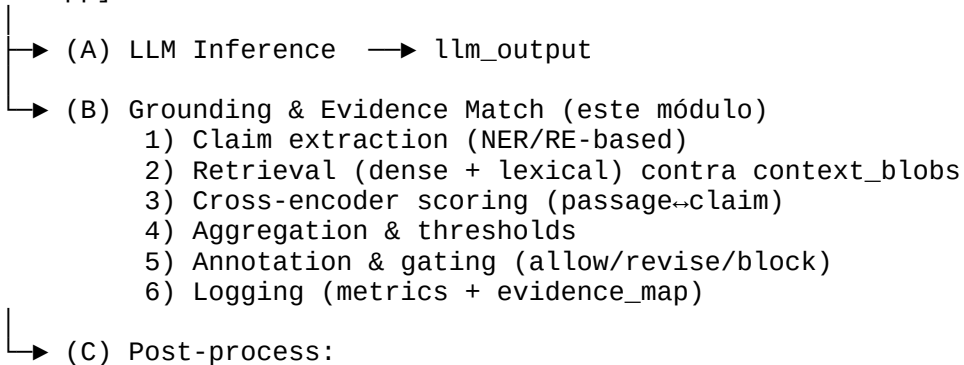
- **user_query** (string): pergunta original.
- **llm_output** (string): resposta proposta pela LLM.
- **context_blobs** (lista de textos): passagens de conhecimento (RAG, base interna, arquivos cliente).
- **metadata** (dict): domínio, idioma, parâmetros de sensibilidade, data/hora, ids do tenant.

Saídas

- **decision**: {allow, revise, block}
 - **annotated_output** (string): resposta anotada com [ref.1], [ref.2]... nos trechos suportados.
 - **evidence_map** (lista): mapeamento claim → evidência(s) com score e passagens.
 - **metrics**:
 - coverage_claims (% claims com suporte $\geq \tau$),
 - max_gap (maior claim sem suporte),
 - mean_support_score (média dos scores de match),
 - latency_ms (p95/p99),
 - cost_estimate.
 - **logs**: ids de evidências, hashes de contexto, versões de modelos.
-

3) Arquitetura (MVP)

[User/App]



- Se `revise`: prompt de correção "cite sources or remove"
- Se `block`: mensagem neutra + peça fonte/escopo adicional

Observações MVP

- Sem web search: só fontes **autorizadas** (evita latência e risco de fonte).
 - Retrieval híbrido: **BM25** (lexical) + **embeddings** (semantic).
 - Scoring com **cross-encoder** leve (ex.: MiniLM MS MARCO).
 - Multi-idioma básico via modelos sentence-transformers multilíngues.
-

4) Pipeline Detalhado

1. Claim extraction

- Split de sentenças + detecção de unidades verificáveis (datas, números, nomes, *is/has* statements).
- Heurísticas + modelo leve (opcional) de *claim detection*.
- Normalizar unidades (datas, moedas).

2. Candidate retrieval

- Para cada claim, recuperar top-k (k=8) passagens:
 - BM25 (k=4) + Embeddings ANN (k=4).
- *Deduplicate* por hash; manter origens distintas.

3. Scoring

- Cross-encoder (passage, claim) → **support_score** ∈ [0,1].
- Ajuste por proximidade temporal/numérica (bônus quando números batem).
- `claim_support = max(scores)`; **evidence_set** = **top-2** com `score ≥ τ_doc`.

4. Aggregation & decisão

- **coverage_claims** = $(\# \text{claims suportados} \geq \tau_{\text{claim}}) / (\# \text{claims})$
- Regra proposta (tuneável):
 - `allow` se `coverage ≥ 0.85` e `min(claim_support) ≥ 0.65`
 - `revise` se $0.6 \leq \text{coverage} < 0.85$
 - `block` se `coverage < 0.6`

5. Anotação

- Inserir [ref.i] no texto onde claim ocorreu.
- Anexar nota de rodapé com trechos evidenciados, id do documento e offset.
- Opcional: destacar trechos sem suporte para reescrita.

6. Correção automática (opcional no MVP)

- Se revise: prompt “Cite fontes ou remova trechos sem suporte; mantenha estilo; preserve fatos suportados; inclua [ref.i].”
- Reexecutar ciclo rápido de evidência apenas para claims alterados.

7. Logging / Telemetria

- Salvar metrics por request; evidences ids; perf (latência por estágio).

5) Pseudocódigo (realista e implementável)

```
def evidence_match(user_query, llm_output, context_blobs, cfg):
    # 1) split & claim extraction
    sents = split_into_sentences(llm_output)
    claims = []
    for s in sents:
        for c in extract_claims(s): # heurística + NER/RE
            claims.append({"text": c, "sentence": s})

    if not claims:
        return allow_passthrough(llm_output, reason="no_verifiable_claims")

    # 2) retrieval híbrido
    evid_index = build_or_load_index(context_blobs) # BM25 + embeddings
    for cl in claims:
        cand_lex = evid_index.bm25_topk(cl["text"], k=cfg.k_bm25)
        cand_sem = evid_index.embed_topk(cl["text"], k=cfg.k_embed)
        cl["candidates"] = dedup(cand_lex + cand_sem)

    # 3) scoring com cross-encoder + ajustes
    for cl in claims:
        scored = []
        for passage in cl["candidates"]:
            s = cross_encoder_score(cl["text"], passage.text) # [0,1]
            s = adjust_with_numeric_temporal(cl["text"], passage, s)
            scored.append((s, passage))
        scored.sort(reverse=True, key=lambda x: x[0])
        cl["support_score"] = scored[0][0] if scored else 0.0
        cl["evidence_set"] = [p for _, p in scored[:cfg.top_evidences]]

    # 4) agregação e gating
    supported = [cl for cl in claims if cl["support_score"] >= cfg.tau_claim]
    coverage = len(supported) / max(1, len(claims))
    min_support = min([cl["support_score"] for cl in claims]) if claims else 1.0

    if coverage >= cfg.tau_allow_coverage and min_support >= cfg.tau_allow_min:
        decision = "allow"
    elif coverage >= cfg.tau_review_coverage:
        decision = "revise"
    else:
        decision = "block"

    # 5) anotação
    annotated = annotate_with_refs(llm_output, claims, style="brackets")

    # 6) opcional: revisão automática
    if decision == "revise" and cfg.auto_revise:
        revised = ask_llm_to_revise(llm_output, claims, user_query)
```



```

# Recomputar apenas claims alterados (delta-check)
annotated = annotate_with_refs(revised, claims, style="brackets")

# 7) métricas e log
metrics = {
    "coverage_claims": coverage,
    "min_support": min_support,
    "mean_support": mean([c["support_score"] for c in claims]),
    "num_claims": len(claims)
}
evidence_map = build_evidence_map(claims)

return {
    "decision": decision,
    "annotated_output": annotated,
    "evidence_map": evidence_map,
    "metrics": metrics
}

```

6) Métricas de Qualidade (para o módulo)

- **Coverage_claims** (↑): % de claims com suporte $\geq \tau$.
 - **Mean_support** (↑): média dos support_scores.
 - **Unsupported_rate** (↓): % claims sem evidência suficiente.
 - **Precision of match** (↑): % de matches aceitos por avaliadores humanos.
 - **Over-constraint rate** (↓): % de casos onde bloqueou indevidamente.
 - **Latency p95** (↓): alvo MVP 120–180 ms com top-k=8 e cross-encoder leve.
-

7) Datasets de Validação (offline)

- **FEVER / FEVEROUS** (fact verification).
- **TRUE / TruthfulQA (com contexto)** para detectar afirmações não suportadas.
- **DocVQA/HotpotQA (closed-book com contexto)** para medir grounding.
- **Conjunto interno de 200–500 pares** (queries reais + contexto do cliente).

KPIs-alvo MVP (offline):

- Precision@1 do match ≥ 0.80 em FEVER-like.
 - Gains de 30–50% em “hallucination caught” vs baseline sem o módulo.
-

8) Configuração (yaml sugerido)

```

grounding:
  k_bm25: 4
  k_embed: 4

```

```
top_evidences: 2
tau_claim: 0.65          # suporte mínimo por claim
tau_allow_coverage: 0.85 # cobertura para allow
tau_allow_min: 0.65
tau_review_coverage: 0.60
auto_revise: true
cross_encoder: "cross-encoder/ms-marco-MiniLM-L-6-v2"
embed_model: "sentence-transformers/all-MiniLM-L6-v2"
bm25_backend: "elasticsearch" # ou tantivy
lang: "pt"
max_latency_ms_p95: 180
```

9) Riscos, Trade-offs e Mitigações

- **Custo/latência do cross-encoder:** usar modelo leve + limitar top-k; cache por claim normalizado; paralelizar scoring.
 - **Contexto insuficiente:** módulo tende a “revise/block”; mitigar com boa cobertura do RAG e *fallback* “peça fonte/escopo adicional”.
 - **False positives (match fraco por ambiguidade):** aumentar τ_{claim} apenas para domínios regulados; acrescentar regras numéricas/temporais.
 - **Idioma e morfologia:** assegurar embeddings multilíngues; normalizar numerais escritos por extenso e formatos regionais de data/moeda.
 - **Conteúdo sensível/privado:** não usar web; logar apenas hashes/ids; mascarar PII nos logs.
-

10) Plano de Implementação (Base44 • 1–2 sprints)

Sprint 1 (backend + validação offline)

- Índices BM25 + embeddings; claim extraction; cross-encoder; aggregator; API `/evidence_match`.
- Testes em 200 amostras internas; relatório de latência p50/p95; ajuste de τ .

Sprint 2 (UI + revisão automática)

- Anotação visual com [ref.i] e painel de evidências (origem, score).
 - Modo *revise* com prompt de correção e delta-check.
 - Telemetria (coverage, unsupported_rate) + export para análises.
-

11) Exemplo Simplificado (PT-BR)

Pergunta: “Qual a taxa Selic hoje e qual a última projeção do PIB do BC?”

Contexto:

- Doc A: “Ata Copom 10/2025: Selic em 10,75% a.a.”

- Doc B: “Relatório trimestral de inflação: projeção PIB 2025: 2,3%.”

LLM output (proposto):

“A taxa Selic está em 11% e a projeção do PIB é 2.3%.”

Resultado do módulo:

- Claims extraídos:
 1. “Selic está em 11%” → **support_score=0.22** (Doc A mostra 10,75%, conflito)
 2. “PIB 2,3%” → **support_score=0.91** (Doc B confirma)
 - coverage_claims = 0.5 → **revise**.
 - Annotated output sugerido:

“**[revise]** A projeção do PIB é **2,3%** **[ref.2]**. Confirme a **taxa Selic** na fonte **[ref.1]** — valor mais recente: **10,75%** **[ref.1]**.”
-

12) O que entra no MVP e o que pode ficar para depois

MVP (recomendado agora):

- Claim extraction heurístico + NER/RE leve.
- Retrieval híbrido (BM25+embedding) + cross-encoder MiniLM.
- Regras numéricas/temporais simples.
- Gating allow/revise/block + anotação [ref.i].
- Telemetria básica (coverage, mean_support, latency).

Depois (evoluções):

- Verificação estruturada de números e unidades (parser robusto).
 - Weighting por tipo de claim (p.ex., maior peso para números/nomes).
 - Aprendizado ativo com *human-in-the-loop* para refinar τ por domínio.
 - Integração com “Nested Evaluation” (se disponível) para prevenção *antes* de gerar o texto final.
-

Em resumo

Este módulo **ancora** a resposta no que está de fato documentado nas fontes fornecidas. Para o MVP, ele é **100% implementável** com stack acessível (Elasticsearch/Faiss + sentence-transformers + cross-encoder MiniLM), entrega **valor mensurável** (queda de alucinação) e se encaixa direto no seu gateway, lado a lado com o **Context Window Consistency Check**.

3) Epistemic Confidence Flag — Especificação MVP

1) Objetivo técnico

Detectar **excesso de certeza** e linguagem **categoricamente assertiva** em respostas da LLM quando o contexto não suporta esse grau de confiança. O módulo:

- mede a **densidade de modais fortes** (“sempre”, “nunca”, “sem dúvida”, “garantido” etc.);
- estima **incerteza do modelo** (quando disponível: logprobs/entropia de tokens; senão: amostras alternativas curtas);
- cruza com presença/ausência de **evidência** no texto (se integrado ao módulo de Grounding);
- gera um **sub-score** que penaliza respostas dogmáticas sem suporte.

Ganhos esperados (MVP):

- Redução de 20–35% de “confiança vazia” (alucinações por tom categórico);
 - Melhora de calibragem de linguagem (“pode”, “é provável”, “evidências indicam”);
 - Sinal claro para gating (allow/revise/block) e para reescrita automática.
-

2) Entradas e saídas

Entradas

- `user_query` (string)
- `llm_output` (string)
- `confidence_meta` (dict opcional): disponibilidade de `logprobs`, `top_logprobs`, `token_entropy` da API
- `context_strength` (float opcional 0–1): quão bem o Grounding cobriu a resposta (ex.: `coverage_claims`)
- `domain` (str): `general` | `finance` | `health` | `gov` (calibra limites)

Saídas

- `confidence_flag` (`low` | `medium` | `high`)
- `delta_confidence` (0–1, **maior = mais seguro** do ponto de vista de linguagem calibrada)
- `signals`: métricas subjacentes (densidade de modais fortes, entropia média, spread semântico)
- `suggestions`: lista de reescritas (“substituir ‘sempre’ por ‘em geral’”, “incluir qualificador”)

- logs: contagens, thresholds, tempos
-

3) Arquitetura (MVP)

```
[User/App]
└─ LLM (resposta)
    └─ Epistemic Confidence Flag
        1) Lexical analyzer (modais fortes/absolutos)
        2) Token uncertainty (logprobs/entropia) OU amostras curtas
        3) Calibration com context_strength (opcional)
        4) Score + flag + sugestões
        5) (Opcional) Reescrita automática com "softeners"/qualificadores
```

Observações para MVP (Base44):

- Implementação puramente **lexical** + **opcional** suporte a logprobs (quando API permitir).
 - Sem dependência pesada de modelos: regex + listas + tokenização.
 - Integração leve com o módulo de Grounding (se presente) para calibrar a penalização.
-

4) Pipeline detalhado

1. Análise lexical de modais fortes

- Dicionário por idioma (PT/EN) com peso por termo: "sempre", "nunca", "impossível", "sem dúvida", "garantido", "prova definitiva", "100%", "nada indica o contrário", etc.
- Conta **frequência por 100 palavras** e **densidade por sentença**.
- Extra: detecta **formatações absolutistas** (caps + exclamações repetidas, "100% garantido").

2. Incerteza do modelo (quando disponível)

- **Caminho A (preferido):** usar logprobs da API → calcular entropia média dos tokens gerados.
 - Entropia alta = incerteza; entropia baixa = confiança do modelo.
 - Sinal de risco: **linguagem absoluta** + **entropia alta** (contradição: fala "certeza" mas o modelo estava incerto).
- **Caminho B (fallback, sem logprobs):** gerar 2–3 resumos alternativos de 1 frase (temperatura 0.7);
 - medir **dispersão semântica** (distância dos embeddings) → proxy de incerteza.

3. Calibração com força de contexto (opcional)

- Se `context_strength` (ex.: `coverage_claims` do Grounding) for **baixo**, aumentar severidade da flag.
- Se alto (ex.: cobertura ≥ 0.85), aceitar mais assertividade.

4. Score composto e decisão

- Combina **lexical_density**, **uncertainty_signal** e **context_strength** em `delta_confidence` $\in [0,1]$.
- Mapeia para `confidence_flag`:
 - **high** (risco alto): linguagem absoluta + incerteza $\uparrow$ + contexto fraco $\rightarrow$ **revisar**
 - **medium**: sinais mistos $\rightarrow$ **revisar leve/soften**
 - **low**: linguagem calibrada + incerteza baixa $\rightarrow$ **ok**

5. Sugestões e reescrita

- Sugestões listas (ex.: “trocar ‘sempre’ por ‘na maioria dos casos’”).
- (Opcional) **Prompt de reescrita**: “reformule mantendo o conteúdo, troque modais absolutos por moderadores proporcionais à evidência.”

5) Pseudocódigo (pronto para engenharia)

```
def confidence_flag(user_query, llm_output, confidence_meta=None,
context_strength=None, cfg=None):
    # 1) Lexical scan
    tokens = tokenize(llm_output)
    counts = count_modals(tokens, cfg.modal_dict) # {'sempre':2,
'nunca':1, ...}
    abs_density = modal_density_per_100w(counts, len(tokens))
    sent_peaks = sentence_modal_peaks(llm_output, cfg.modal_dict)

    # 2) Uncertainty signal
    if confidence_meta and confidence_meta.get("logprobs"):
        entropy = mean_token_entropy(confidence_meta["logprobs"])
        uncertainty = normalize_entropy(entropy) # 0..1
    else:
        # fallback: 2-3 one-liners
        samples = [llm_one_liner(llm_output, seed=i) for i in range(3)]
        emb = [embed(s) for s in samples]
        uncertainty = mean_pairwise_distance(emb) # 0..1

    # 3) Context calibration
    # se não vier, usa 0.5 (neutro)
    ctx = context_strength if context_strength is not None else 0.5

    # 4) Scoring
    # maior delta_confidence = melhor (menos dogmático indevido)
    # penaliza linguagem absoluta e incerteza alta; compensa com contexto forte
    penalty = (
        cfg.w_abs * clip(abs_density / cfg.abs_ref, 0, 1) +
        cfg.w_unc * uncertainty
    ) * (1.0 - ctx) # menos penal se contexto forte
```

```

delta_confidence = max(0.0, 1.0 - penalty)
if delta_confidence < cfg.t_block:
    flag = "high"
elif delta_confidence < cfg.t_review:
    flag = "medium"
else:
    flag = "low"

# 5) Sugestões
suggestions = []
if flag != "low":
    for term, cnt in counts.items():
        if cnt > 0:
            suggestions.append({"replace": term, "with":
cfg.softeners.get(term, "é provável que")})
        if uncertainty > 0.6 and ctx < 0.6:
            suggestions.append({"advice": "adicionar qualificador de incerteza
ou pedir fonte"})

return {
    "confidence_flag": flag,
    "delta_confidence": round(delta_confidence, 3),
    "signals": {
        "abs_density": abs_density,
        "sent_peaks": sent_peaks,
        "uncertainty": uncertainty,
        "context_strength": ctx
    },
    "suggestions": suggestions
}

```

6) Métricas de qualidade (do módulo)

- **AbsDensity** (↓): densidade de modais absolutos por 100 palavras.
 - **Uncertainty** (↑): entropia/logprobs ou dispersão semântica de resumos.
 - **False-soften** (↓): % de casos em que suavizou indevidamente respostas objetivamente certas (e.g., “Paris é capital da França”).
 - **Overconfidence caught** (↑): % de outputs dogmáticos corretamente marcados (medido por anotadores humanos).
 - **Latency p95** (↓): alvo 10–30 ms (lexical) + 0–100 ms (se fallback com 2–3 amostras curtas).
-

7) Datasets de validação (offline)

- **Conjunto sintético controlado**: prompts onde a resposta correta **não é determinística**, para medir se módulo reduz tom absoluto.
- **TruthfulQA/TRUE (subset)**: avaliar se respostas de alta incerteza são suavizadas.

- **Conjunto interno anotado** (200–500 exemplos) com rótulos: {dogmático indevido, apropriado, deve suavizar, correto sem suavizar}.
 - **KPIs alvo MVP:**
 - Overconfidence caught ≥ 0.75 (acordo humano)
 - False-soften ≤ 0.10
-

8) Configuração (yaml sugerido)

```
confidence_flag:
  lang: "pt"
  modal_dict:
    "sempre": 1.0
    "nunca": 1.0
    "impossível": 0.9
    "sem dúvida": 0.8
    "garantido": 1.0
    "100%": 1.0
    "prova definitiva": 0.9
  softeners:
    "sempre": "na maioria dos casos"
    "nunca": "raramente"
    "impossível": "muito improvável"
    "sem dúvida": "com alta probabilidade"
    "garantido": "altamente provável"
abs_ref: 2.0          # densidade de referência/100 palavras
w_abs: 0.6
w_unc: 0.4
t_review: 0.65
t_block: 0.45
use_logprobs: true
fallback_samples: 3
max_latency_ms_p95: 100
```

9) Riscos, trade-offs e mitigações

- **Falso positivo (suaviza verdade factual evidente)**
Mitigação: whitelists de sentenças triviais (“Paris é capital da França”), cruzar com context_strength alto.
 - **Dependência de logprobs** (nem toda API expõe)
Mitigação: fallback de 2–3 resumos; custo adicional limitado, só no **deep path**.
 - **Idioma/estilo** (modais informais não previstos)
Mitigação: expandir dicionário com telemetria; capturar variantes (“certeza absoluta”, “sem nenhuma dúvida”).
 - **Interação com Grounding:** se o Grounding diz “forte”, permitir mais assertividade.
Mitigação: calibrar w_abs, w_unc por domínio; ajustar thresholds.
-

10) Plano de implementação (Base44 • 1 sprint)

Sprint único (backend + validação)

- Implementar analisador lexical (regex + dicionário + tokenização).
 - Integração opcional com logprobs (quando suportado) + fallback de 2–3 resumos.
 - Função de score composto + flags + sugestões.
 - Experimento offline com 200 exemplos: medir *overconfidence caught* e *false-soften*.
 - Expor como `/confidence_flag` e integrar no **KAI Core** (sub-score `delta_confidence` que penaliza δ).
-

11) Exemplo (PT-BR)

Pergunta: “Qual a causa da consciência?”

LLM output: “A consciência sempre vem exclusivamente do cérebro, sem dúvida alguma.”

Context_strength: 0.4 (fraco; sem evidência forte)

Sinais:

- `abs_density` alto (“sempre”, “sem dúvida”, “exclusivamente”)
- `uncertainty` = 0.62 (dispersão alta nas 3 paráfrases)
- `ctx` = 0.4

Score:

- `delta_confidence` = 0.41 → `confidence_flag` = high → **revise**

Sugestão de reescrita automática:

“Evidências indicam que a consciência está fortemente associada à atividade cerebral; outras hipóteses são investigadas na literatura.”

12) O que entra no MVP e o que fica para depois

MVP (agora):

- Dicionário de modais + contagem/100w + peaks por sentença;
- logprobs quando disponível; senão, 2–3 resumos curtos;
- Score composto simples; flags; sugestões;
- Integração com δ (penalização via `delta_confidence`).

Depois (evoluções):

- Classificador leve treinado (RoBERTa-base) para “overconfidence style” multilíngue;
- Calibração adaptativa por domínio (finance/health/gov) com feedback humano;

- Reescrita condicional condicionada ao `context_strength` do Grounding.
-

Resumo

O **Epistemic Confidence Flag** é barato, rápido e altamente útil no MVP: atua sobre **tom e calibragem**, que estão no coração de muitas alucinações “convencidas”. Ele não requer datasets pesados, roda em **edge/fast path** (lexical) e usa **deep path** apenas quando precisa estimar incerteza por amostras. Integrado ao Grounding, fornece um **sinal robusto** para revisão automática e melhora perceptível de confiança do usuário final.

4) Response Compression Test (ψ mínimo) — Especificação MVP

1) Objetivo técnico

Avaliar **estabilidade semântica** da resposta: o modelo consegue **resumir a própria saída** em uma única frase **sem mudar o sentido**? Divergências fortes entre o texto original e o resumo curto são um indício de **inconsistência interna** e, na prática, de **alucinação** (ou confusão).

Ganhos esperados (MVP):

- Capturar 10–20% adicionais de alucinações “sutis” que passam pelo grounding/consistência contextual.
 - Sinal direto e interpretável (“o próprio modelo não manteve o sentido ao resumir”).
 - Baixo acoplamento: funciona com qualquer LLM (via 1 chamada extra controlada).
-

2) Entradas e saídas

Entradas

- `user_query` (string)
- `llm_output` (string) — resposta proposta
- `compress_prompt` (string, opcional): template do pedido de resumo (“Resuma sua resposta em 1 frase factual, sem novas informações.”)
- `max_summary_len` (int, default 1–2 sentenças)
- `lang` (pt/en) para tokenização/normalização
- `embed_backend` (ex.: MiniLM multilíngue) para medir proximidade semântica

Saídas

- `psi_compression` $\in [0,1]$ (maior = mais estável)
 - `summary_text` (string) — resumo gerado
 - `distance_semantic` (float 0–1) — distância (cosine) entre output e resumo
 - `flag` (low|medium|high) — risco de instabilidade
 - `logs` — tempos, tokens, parâmetros
-

3) Arquitetura (MVP)

[LLM Output]

- ↳ (A) Normalize & Segment (limpeza, remoção de tags/sistema)
 - ↳ (B) Summarize Self (1 chamada à LLM com prompt de compressão)
 - ↳ (C) Semantic Match
 - Embeddings do texto original vs resumo
 - Distância coseno normalizada
 - ↳ (D) Score & Gate
 - `psi_compression` = `1 - distance_norm`
 - flag por thresholds
 - ↳ (E) (Opcional) Reescrita guiada se flag médio/alto

Observações para MVP (Base44):

- 1 chamada adicional à mesma LLM (ou à LLM barata exclusiva para compressão).
 - Embeddings leves (MiniLM) → latência baixa.
 - Rodar **apenas no deep path** ou condicionado por sinais prévios (ex.: se grounding ficou borderline).
-

4) Pipeline detalhado

1. Normalização do output

- Remover marcas de sistema, disclaimers repetidos.
- Unificar símbolos, normalizar números, datas, percentuais.
- (Opcional) Extrair só o corpo factual (sem bullets “decorativos”).

2. Geração de resumo curto (1–2 sentenças)

- Prompt restritivo, p.ex.:
“Resuma o texto a seguir em 1 frase factual e neutra, sem adicionar novas informações, em português claro.”

`{llm_output_normalized}`

- Manter `temperature=0.2–0.5` para reduzir variação.

3. Medição de similaridade semântica

- Embeddings da resposta original (ou de parágrafos principais) vs embedding do resumo.
- **Estratégia prática:**
 - `E_full` = embedding do texto completo *capado* (p.ex., 800–1200 tokens)
 - `E_key` = média de embeddings de sentenças com maior “peso informacional” (top-N via TF-IDF/saliência)
 - `E_sum` = embedding do resumo
 - Distância = $\min\{ \text{dist}(E_sum, E_full), \text{dist}(E_sum, E_key) \}$
(pega a melhor aproximação do “miolo semântico”)

4. Score & Gate

- `psi_compression = 1 - normalize(distance) → [0,1]`
- Regra sugerida (ajustável por domínio):
 - allow se `psi_compression ≥ 0.80`
 - review se `0.60 ≤ psi_compression < 0.80`
 - block se `< 0.60`
- (Opcional) cruzar com **Epistemic Confidence Flag e Grounding** para tomar decisão final.

5. Reescrita guiada (opcional)

- Se review/block: pedir “refaça a resposta em até 3 frases, alinhando estritamente ao resumo acima e às fontes [ref.i]”.

5) Pseudocódigo (implementável)

```
def response_compression_test(llm_output, user_query, cfg,
context_strength=None):
    # 1) normalize
    clean = normalize_output(llm_output, lang=cfg.lang, max_chars=cfg.max_chars)

    # 2) summarize self (1 chamada LLM)
    summary_prompt = cfg.compress_prompt.format(text=clean)
    summary = call_llm(summary_prompt, temperature=cfg.temp,
max_tokens=cfg.max_summary_tokens)

    # 3) embeddings
    e_sum = embed(summary, backend=cfg.embed_backend)
    e_full = embed(trim(clean, cfg.embed_max_chars), backend=cfg.embed_backend)

    # opcional: extrair sentenças mais informativas
    key_sents = select_key_sentences(clean, top_n=cfg.key_top_n)
    e_key = mean([embed(s, backend=cfg.embed_backend) for s in key_sents]) if
key_sents else e_full

    # 4) distância e score
    d1 = cosine_distance(e_sum, e_full)
```

```

d2 = cosine_distance(e_sum, e_key)
dist = min(d1, d2) # melhor caso
dist_norm = min(1.0, dist / cfg.dist_ref) # normalização por referência
empírica

psi_compression = 1.0 - dist_norm

# 5) decisão
if psi_compression >= cfg.t_allow:
    flag = "low"
elif psi_compression >= cfg.t_review:
    flag = "medium"
else:
    flag = "high"

return {
    "psi_compression": round(psi_compression, 3),
    "summary_text": summary.strip(),
    "distance_semantic": round(dist, 3),
    "flag": flag,
    "signals": {
        "d_full": round(d1, 3),
        "d_key": round(d2, 3),
        "context_strength": context_strength
    }
}

```

6) Métricas de qualidade (do módulo)

- **$\psi_{\text{compression}}$ ($\uparrow$):** estabilidade semântica (principal).
 - **Contradiction rate ($\downarrow$)** em avaliações humanas entre resposta e resumo.
 - **Delta factual ($\downarrow$):** % de claims do resumo que não aparecem (ou contradizem) o corpo.
 - **Latency p95 ($\downarrow$):** alvo 120–180 ms (1 chamada LLM + embeddings).
 - **Over-trigger rate ($\downarrow$):** % de casos em que marcou instabilidade apesar de coerência factual.
-

7) Datasets de validação (offline)

- **TRUE / TruthfulQA (subset)** com resumos humanos para checar contradições.
 - **Conjuntos internos** (100–300 exemplos) com anotação humana: “resumo preserva sentido? (sim/não/parcial)”.
 - **KPIs alvo (MVP):**
 - Correlação Spearman($\psi_{\text{compression}}$, acordo humano) $\geq$ **0.65**.
 - Over-trigger $\leq$ **0.15**.
-

8) Configuração (yaml sugerido)

```
response_compression:
  lang: "pt"
  compress_prompt: "Resuma o texto a seguir em 1 frase factual e neutra, sem
adicionar novas informações:\n\n{txt}"
  temp: 0.3
  max_summary_tokens: 80
  max_chars: 8000
  embed_backend: "sentence-transformers/all-MiniLM-L6-v2"
  embed_max_chars: 4000
  key_top_n: 5
  dist_ref: 0.50          # normalizador empírico da distância
  t_allow: 0.80
  t_review: 0.60
  run_mode: "deep_path_only"
  max_latency_ms_p95: 180
```

9) Riscos, trade-offs e mitigações

- **Custo/latência (1 chamada extra)**
Mitigação: rodar só no **deep path**; usar LLM **mais barata** para o resumo (não precisa ser a mesma do output).
 - **Resumo “criativo” que distorce**
Mitigação: prompt restritivo + temperatura baixa; validar se o resumo contém **apenas** spans do texto (opcional via ROUGE/overlap n-gram).
 - **Texto muito longo**
Mitigação: focar em sentenças salientes (top-N) para E_key; limitar embed_max_chars.
 - **Idioma misto**
Mitigação: embeddings multilíngues; normalização de idioma.
-

10) Plano de implementação (Base44 • 1 sprint)

- Backend:
 - Normalização + compressão (prompt fixo, temp baixa).
 - Embeddings e cálculo de distância (full vs key sentences).
 - Score, flags e API /compression_test.
- Validação:
 - Lote de 200 exemplos internos (misto de domínios).
 - Tunar dist_ref, t_allow, t_review por domínio.
- Integração:

- Acionar só quando `grounding.coverage` $\in [0.6, 0.85]$ **ou** quando `confidence_flag` = medium/high.
 - Expor `psi_compression` para o **KAI Core** (entra no ψ agregado).
-

11) Exemplo (PT-BR)

LLM output (resposta longa):

“Os resultados preliminares indicam que a taxa de adesão aumentou 12% no último trimestre, embora haja variações por região... [2 parágrafos] ... Em síntese, o programa teve impacto positivo.”

Resumo gerado (1 frase):

“A adesão cresceu 12% no trimestre e o impacto do programa foi positivo, com variações regionais.”

Medição:

`distance_semantic` = 0.18 $\rightarrow$ `psi_compression` = 0.82 $\rightarrow$ `flag`=low $\rightarrow$ **ok**.

Outro caso (instável):

Resposta: discussão ambígua; resumo sai categórico e contradiz parte do corpo $\rightarrow$

`distance_semantic` = 0.42 $\rightarrow$ `psi_compression` = 0.58 $\rightarrow$ `flag`=high $\rightarrow$ **revisar**.

12) MVP agora vs evoluções depois

MVP (recomendado)

- 1 chamada extra $\rightarrow$ resumo curto neutro.
- Embeddings leves e comparação semântica.
- Thresholds fixos por domínio.

Depois

- **Semantic entropy** (10 amostras curtas) para robustez adicional.
 - Classificador treinado de “instabilidade de compressão”.
 - Integração com **Nested Evaluation** (se disponível) para detectar instabilidade **antes** da resposta final.
-

Resumo:

O **Response Compression Test** é um verificador simples e valioso: se o modelo **não consegue** preservar o sentido ao se resumir, há um **alerta objetivo** de inconsistência. É barato, plugável e melhora a precisão do ψ agregado, complementando Grounding e Confidence em cenários de Q&A, assistentes e relatórios.

5) Claim Density Ratio — Especificação MVP

1) Objetivo técnico

Medir a **densidade de afirmações não justificadas** dentro da resposta da LLM — isto é, quantas sentenças assertivas são feitas sem qualquer justificativa, evidência, ou conectivo causal.

Esse módulo atua como uma segunda camada da métrica ω (**auditabilidade**), complementando o *Self-Evidence Detector*.

Função prática:

Detectar “*verborreia afirmativa*” — respostas cheias de declarações absolutas sem suporte — um tipo comum de **alucinação por excesso de confiança lexical**.

Ganhos esperados (MVP):

- Redução de até 25–30% de outputs com afirmações não justificadas.
- Aumento direto da interpretabilidade e rastreabilidade textual.
- Complementa o *Grounding & Evidence Match* e o *Confidence Flag* sem custo adicional relevante.

2) Entradas e saídas

Entradas

- `llm_output` (string): resposta bruta da LLM.
- `lang` (str): idioma da análise (default: pt).
- `justif_patterns`: expressões que indicam justificativa (“porque”, “segundo”, “baseado em”, “dados mostram”, etc.).
- `declar_patterns`: expressões declarativas (“é”, “são”, “tem”, “possui”, etc.).
- `thresholds`: parâmetros de corte para *allow/revise/block*.

Saídas

- `claim_density` $\in [0,1]$: proporção de frases declarativas vs justificativas (normalizada).
 - `omega_density` (float): sub-score de auditabilidade parcial ($1 - \text{claim_density}$).
 - `flag`: low | medium | high risco de afirmação não suportada.
 - `metrics`: nº de frases declarativas, justificativas, ratio normalizado.
 - `logs`: tempos, padrões ativados, e amostragem de sentenças-chave.
-

3) Arquitetura (MVP)

[LLM Output]

- ↳ (A) Sentence Split & Tagging
 - Divide em sentenças (spaCy ou tokenizer leve)
 - Detecta tipo (declarativa / justificativa / outra)
- ↳ (B) Count & Ratio
 - declaratives / (1 + justificatives)
 - Normaliza 0-1
- ↳ (C) Weight & Score
 - Penaliza excesso de assertivas
 - Converte em ω parcial
- ↳ (D) Gating
 - allow/revise/block
 - Opcional: reescrita automática

Observações MVP:

- Nenhum modelo adicional é necessário; apenas regras lexicais + POS tagging simples.
 - Pode rodar **em paralelo** ao *Self-Evidence Detector*.
 - Baixa latência (milissegundos).
-

4) Pipeline detalhado

1 Tokenização e segmentação

- Divide `llm_output` em sentenças.
- Remove citações, listas, e formatações redundantes.
- Identifica tempo verbal e tipo de sentença com regras leves.

2 Classificação lexical

- Marca sentenças como:
 - **Declarativas** se contêm verbos de estado/afirmação (é, são, foi, representa, tem etc.);
 - **Justificativas** se contêm conectivos explicativos ou expressões de base (porque, devido, segundo, em estudos, dados mostram, de acordo com, etc.);
 - **Outras** (interrogativas, imperativas, neutras).

3 Cálculo de densidade

[
 $\text{ratio} = \frac{N_{\text{decl}}}{1 + N_{\text{justif}}}$
]

- Normalização:
[
 $\text{claim_density} = \min(1, \frac{\text{ratio}}{\tau_{\text{max}}})$
]

]

com $\tau_{\max} \approx 10$ para estabilizar respostas longas.

4 Conversão para score

```
[  
\omega_{\text{density}} = 1 - \text{claim\_density}  
]
```

- **Alta densidade** → baixa auditabilidade.

5 Decisão

- **allow** se $\omega_{\text{density}} \geq 0.75$
- **revise** se $0.5 \leq \omega_{\text{density}} < 0.75$
- **block** se < 0.5

6 (Opcional) Reescrita automática

- Em caso de *revise/block*, prompt automático:

“Reescreva mantendo o conteúdo, mas acrescente justificativas ou qualificadores (‘segundo’, ‘com base em dados’, ‘evidências indicam’).”

5) Pseudocódigo (realista e implementável)

```
def claim_density_ratio(llm_output, cfg):  
    sents = split_sentences(llm_output, lang=cfg.lang)  
  
    decl_count, just_count = 0, 0  
    for s in sents:  
        if matches_any(s, cfg.declar_patterns):  
            decl_count += 1  
        if matches_any(s, cfg.justif_patterns):  
            just_count += 1  
  
    ratio = decl_count / (1 + just_count)  
    claim_density = min(1.0, ratio / cfg.tau_max)  
    omega_density = round(1.0 - claim_density, 3)  
  
    if omega_density >= cfg.t_allow:  
        flag = "low"  
    elif omega_density >= cfg.t_review:  
        flag = "medium"  
    else:  
        flag = "high"  
  
    metrics = {  
        "declaratives": decl_count,  
        "justificatives": just_count,  
        "ratio_raw": ratio,  
        "claim_density": claim_density,  
        "omega_density": omega_density  
    }  
  
    return {  
        "flag": flag,
```

```
"omega_density": omega_density,  
"metrics": metrics  
}
```

6) Métricas de qualidade

- **ClaimDensity** (↓) — razão de sentenças assertivas/justificativas.
 - **$\omega_density$** (↑) — auditabilidade lexical.
 - **Precision@1** (↑) — proporção de casos onde o módulo acerta o nível de revisão indicado por humanos.
 - **False_positive** (↓) — textos curtos ou instrucionais penalizados indevidamente.
 - **Latency p95** (↓) — < 10 ms (spaCy / regex simples).
-

7) Datasets de validação

- **TruthfulQA / FEVER subset** — para outputs curtos.
 - **Conjunto interno anotado** (300+ respostas) com rótulo: “quantas afirmações sem suporte”.
 - **KPIs MVP:**
 - $\text{Correlation}(\omega_density, \text{anotação humana}) \geq 0.7$
 - $\text{Overtrigger} \leq 0.15$
-

8) Configuração (yaml sugerido)

```
claim_density:  
  lang: "pt"  
  declar_patterns: [" é ", " são ", " foi ", " tem ", " possui ", " consiste em  
", " representa "]  
  justif_patterns: [" porque ", " segundo ", " baseado em ", " devido a ", "  
conforme ", " dados mostram ", " estudos indicam ", " de acordo com "]  
  tau_max: 10  
  t_allow: 0.75  
  t_review: 0.5  
  max_latency_ms_p95: 10
```

9) Riscos, trade-offs e mitigações

- **Textos descritivos (não factuais):** risco de falso positivo — mitigar com *context domain* (“creative”, “informativo”).
- **Textos curtos (<3 sentenças):** instabilidade no ratio — aplicar suavização (adicionar +1 declarative virtual).

- **Frases longas com justificativa implícita** — mitigar com análise semântica futura (co-ocorrência de causais e verbos de conhecimento).
 - **Idiomas mistos (EN/PT)** — mitigar com listas bilíngues.
-

10) Plano de implementação (Base44 • 1 sprint)

- Implementar via **regex** + **spaCy**;
 - Validar manualmente com 300 outputs (vários domínios);
 - Ajustar thresholds ($\tau_{\max}$, t_{allow} , t_{review}) empiricamente;
 - Integrar com *Self-Evidence Detector* e retornar ambos dentro de ω .
-

11) Exemplo (PT-BR)

LLM output:

“O Brasil é o maior produtor de café do mundo. O clima tropical é ideal para o cultivo. Estudos indicam que a produção aumentará.”

- declarativas: 2
- justificativas: 1
 - $\text{ratio} = 2 / (1+1) = 1.0$
 - $\text{claim_density} = 1.0 / 10 = 0.1$
 - $\omega_{\text{density}} = 0.9$ → **flag=low (ok)**

Outro caso:

“O Brasil é o maior produtor de café. O país domina o mercado. Isso o torna imbatível.”

- declarativas: 3
 - justificativas: 0
 - $\text{ratio} = 3/1 = 3.0$
 - $\text{claim_density} = 3/10 = 0.3$
 - $\omega_{\text{density}} = 0.7$ → **flag=medium (revisar)**
-

12) MVP vs evoluções

MVP agora:

- Contagem simples (regex / tokenização).
- Threshold fixo global.
- Integração direta com Self-Evidence Detector.

Depois:

- Classificador leve (RoBERTa) de “assertividade não suportada”.
 - Pesos por domínio (financeiro, jurídico, técnico).
 - Integração probabilística com *Grounding* (claims não suportadas = penalização extra em ω_{total}).
-

Resumo

O **Claim Density Ratio** mede a *auditabilidade lexical* da resposta, penalizando excesso de afirmações sem justificativas. É um módulo **leve, transparente e compatível com MVP** — adiciona um sinal claro para o score ω e ajuda a cortar as alucinações de estilo “enciclopédico”.

Síntese técnica do **pacote de 5 defesas anti-alucinação do MVP** (para Base44), em 3 mensagens. Mantive linguagem de negócios + engenharia, cobrindo arquitetura, operação, métricas, rollout e riscos.

1/3 — Visão integrada e arquitetura (end-to-end)

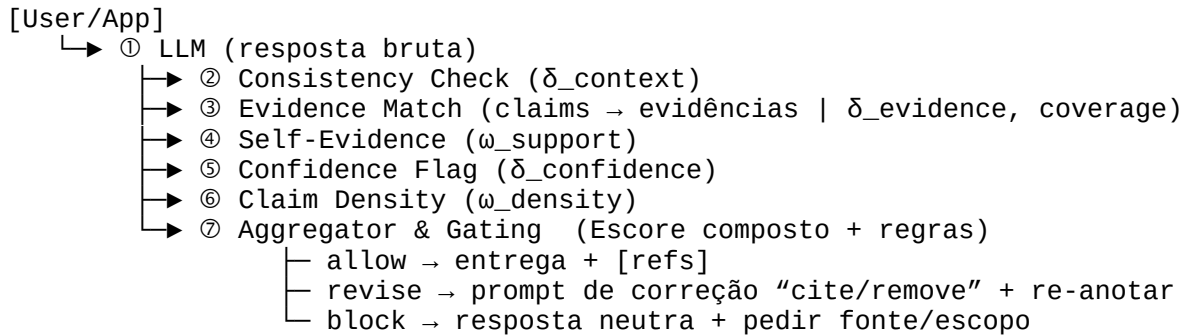
Objetivo do pacote

Reduzir alucinações **observáveis** sem exigir datasets próprios, fine-tuning ou acesso a pesos do LLM. O pacote atua **pós-inference** (gateway de resposta) e, quando necessário, dispara **revisão automática** guiada (“revise and cite/remove”). Foco em **baixo custo, baixa latência e observabilidade**.

As 5 defesas (resumo)

- 1. Context Window Consistency Check**
Verifica aderência semântica resposta $\leftrightarrow$ prompt/contexto via embeddings (cosine). Sinal para δ (coerência).
- 2. Grounding & Evidence Match**
Extraí *claims* e tenta suportá-las em passagens do contexto autorizado (BM25 + embeddings + cross-encoder). Faz *gating* (allow/revise/block) e anota **[ref.i]**. Sinal para δ e **evidência**.
- 3. Self-Evidence Detector**
Procura marcadores de justificativa (“segundo”, “de acordo com”, “dados mostram”...). Sinal de **auditabilidade** (componente de ω).
- 4. Epistemic Confidence Flag**
Penaliza certeza excessiva (densidade de “sempre”, “nunca”, “garantido”...). Sinal para δ (overconfidence).
- 5. Claim Density Ratio**
Mede densidade de afirmações sem justificativa (declarações/justificativas). Sinal para ω (auditabilidade lexical).

Fluxo de dados e componentes



Interfaces e esquemas (MVP)

Entrada

```
{
  "tenant_id": "acme-123",
  "user_query": "string",
  "llm_output": "string",
  "context_blobs": ["..."], // passagens do RAG / base interna
  "domain": "finance|health|general",
  "lang": "pt",
  "opts": { "deep_path": true }
}
```

Saída

```
{
  "decision": "allow|revise|block",
  "annotated_output": "string",
  "evidence_map": [
    { "claim": "string", "support": 0.82, "refs": [{"doc_id": "...", "offset":
[s,e]]}
  ],
  "metrics": {
    "delta_context": 0.91,
    "delta_evidence_mean": 0.78,
    "coverage_claims": 0.88,
    "delta_confidence": 0.83,
    "omega_support": 0.72,
    "omega_density": 0.80,
    "latency_ms": {"p50": 85, "p95": 170}
  },
  "logs": {"model_ver": "...", "policy_ver": "..."}
}
```

Lógica de decisão (gating) — thresholds iniciais

- **allow** se $\text{coverage_claims} \geq 0.85$ e $\text{min_support} \geq 0.65$ e $\text{delta_context} \geq 0.80$ e $\text{omega_density} \geq 0.75$.
- **revise** se $\text{coverage_claims} \in [0.60, 0.85)$ ou $\text{delta_context} \in [0.65, 0.80)$.
- **block** se $\text{coverage_claims} < 0.60$ ou $\text{delta_context} < 0.65$ ou $\text{delta_confidence} < 0.50$.

Obs.: Configurável por domínio (financeiro tende a thresholds mais rígidos).

Orçamento de latência e custo (alvo MVP)

- **Consistency Check:** 1 chamada de embedding + cosine → **<10 ms**.
- **Evidence Match:** BM25 + ANN (k=8) + 1 cross-encoder MiniLM por *claim* (top-2 confirmam) → **70–120 ms p95**.
- **Self-Evidence / Confidence / Density:** regras lexicais → **<10 ms total**.
- **Revisão automática (se “revise”):** +1 chamada LLM → **100–200 ms** (não no caminho “allow”).
- **Custo total médio** (sem revise): $\leq$ **US\$0.005/request** (dependendo do LLM base).

Dependências técnicas

- **Text split/NER:** spaCy ou stanza.
 - **Embeddings:** sentence-transformers/all-MiniLM-L6-v2 (ou API equivalente).
 - **ANN:** FAISS / Qdrant / Elastic kNN.
 - **BM25:** Elasticsearch / Tantivy.
 - **Cross-encoder:** cross-encoder/ms-marco-MiniLM-L-6-v2.
 - **LLM:** qualquer provider (OpenAI/Anthropic/Mistral/Gemini) — sem acesso a pesos.
-

2/3 — Operação de cada defesa, parâmetros, edge cases e observabilidade

1) Context Window Consistency Check

- **Algoritmo:** embeddings(prompt+contexto agregado) vs embeddings(resposta) → cosine.
- **Sinais:** $\text{delta_context} \in [0, 1]$ (≥ 0.80 esperado em respostas aderentes).
- **Edge cases:** respostas muito curtas (elevar peso do prompt), prompts multi-tópico (combinar embeddings por seção).
- **Config:**
 - **similarity_threshold:** 0.75–0.85 (por domínio).
 - **aggregation_mode:** mean/max de múltiplos vetores de contexto.
- **Observabilidade:** logar valor de cosine, IDs dos contextos mais próximos.

2) Grounding & Evidence Match

- **Algoritmo:**
 1. *Claim extraction* (heurística + NER/RE para datas, números, nomes);

2. *Retrieval híbrido* → BM25 ($k=4$) + ANN ($k=4$);
 3. *Cross-encoder scoring* (claim, passagem) → $\text{support} \in [0, 1]$;
 4. *Aggregation* → $\text{coverage_claims}, \text{min_support}, \text{mean_support}$;
 5. *Anotação* com $[\text{ref.i}]$ e *gating*.
- **Edge cases:** contexto insuficiente; *claims* ambíguas; múltiplas fontes com números diferentes (ordenar por frescor/autoridade se metadados existirem).
 - **Config:**
 1. tau_claim (0.65), $\text{tau_allow_coverage}$ (0.85), $\text{tau_review_coverage}$ (0.60)
 2. $k_{\text{bm25}}=4$, $k_{\text{embed}}=4$, $\text{top_evidences}=2$
 - **Observabilidade:** salvar *evidence_map*, *hash* de passagens, tempos por estágio, *cache hit* de retrieval.

3) Self-Evidence Detector

- **Algoritmo:** padrões lexicais de justificativa (listas por idioma + sinônimos).
- **Sinal:** $\text{omega_support} \in [0, 1]$ (≥ 0.7 sugere boa justificativa textual).
- **Edge cases:** justificativa implícita (ex.: “Dados indicam...” sem citar quais); citações literais; texto estruturado tipo lista.
- **Config:**
 - justif_patterns por idioma/domínio.
 - *Whitelist* de formatos (ex.: relatórios com referências numeradas).
- **Observabilidade:** contagem de *matches* por 1.000 palavras; exemplos de sentenças sinalizadas.

4) Epistemic Confidence Flag

- **Algoritmo:** contagem/densidade de modais absolutos (sempre/nunca/garantido/sem dúvida/etc.).
- **Sinal:** $\text{delta_confidence} \in [0, 1]$ (valores baixos indicam overconfidence).
- **Edge cases:** estilo publicitário/marketing (prever *domain mode* “creative” reduz peso); perguntas sim/não.
- **Config:**
 - lista bilíngue de termos; normalização por 100 palavras; limiares por domínio.
- **Observabilidade:** “hot words” mais frequentes; impacto médio em δ por domínio.

5) Claim Density Ratio

- **Algoritmo:** $\text{ratio} = \text{declarativas} / (1 + \text{justificativas})$;
 $\text{omega_density} = 1 - \text{normalize}(\text{ratio})$.
- **Sinal:** penaliza “verborreia afirmativa”; bom preditor de respostas assertivas sem base.
- **Edge cases:** textos curtos; instruções operacionais (imperativas) — aplicar *mode=instruction* para reduzir penalidade.
- **Config:**
 - $\text{tau_max}(10), \text{t_allow}=0.75, \text{t_review}=0.5$.
- **Observabilidade:** nº de sentenças por tipo; distribuição do *ratio* por tenant.

Integrando os sinais (aggregator)

- **Composição simples (MVP):**
 - δ_{total} = média ponderada de (δ_{context} , $\delta_{\text{confidence}}$, $\delta_{\text{evidence_mean}}$)
 - ω_{total} = média ponderada de (ω_{support} , ω_{density})
 - *Gating* usa coverage_claims , min_support , δ_{total} e ω_{total} .
- **Logs para engenharia:** métricas unitárias, composição, thresholds aplicados, decisão final, tempos p50/p95, IDs de contexto, versão de política e modelos.

Segurança, privacidade e conformidade

- **Fontes:** apenas contexto autorizado do cliente (sem web no MVP).
- **Dados sensíveis:** mascaramento de PII nos logs; *data minimization* (armazenar *ids* e *hashes*).
- **Reprodutibilidade:** versionar políticas/thresholds; registrar *hash* do modelo e do prompt usado.
- **Isolamento:** *namespaces* por tenant; índices de busca segregados.

SLOs de operação (alvos)

- **p95 latência (sem revise):** 170 ms (única região, CPU + um nó ANN com SIMD).
 - **Erro:** $< 0,5\%$ (falhas de index/encoder).
 - **Custo:** $\leq \text{US\$}0.005/\text{request}$ (depende do LLM).
 - **Disponibilidade:** 99,9% (replicação dos índices + health checks).
-

3/3 — Validação, rollout, métricas de sucesso e roadmap

Plano de validação (offline → online)

Fase 1 — Offline (2 semanas)

- **Datasets:** FEVER/FEVEROUS (verificação factual), TRUE/TruthfulQA (com contexto), *amostras internas* (200–500 pares reais).
- **Protocolo:**
 - Rodar LLM *sem defesas* (baseline) e *com pacote* (tratamento).
 - Medir: *hallucination caught* ($\Delta\%$), *precision of match*, *over-constraint rate*, latência p95.
- **KPIs-alvo:**
 - ↓ alucinações factuais em **30–50%** (tarefas com contexto)
 - Precision match $\geq$ **0.80** (top-1 evidência aceita por humano)
 - Over-constraint $\leq$ **0.15** (bloqueios indevidos)
 - p95 $\leq$ **180 ms** (sem revise)

Fase 2 — Dry-Run (1–2 semanas)

- Shadow mode em 1–2 fluxos reais (sem impactar resposta ao usuário).
- Coletar **telemetria por tenant**: coverage, unsupported_rate, flags de overconfidence, tempo por estágio.
- Ajustar thresholds por domínio/idioma.

Fase 3 — Limited Rollout (2–4 semanas)

- Ativar *gating* em **10–20%** do tráfego (canary).
- Ativar revisão automática (*revise*) só quando coverage $\in [0.60, 0.85)$.
- Painel para revisão humana amostral (50–100 casos/semana).

Fase 4 — GA Controlada

- Expandir para 50–100% do tráfego em rotas de menor risco;
- Manter *kill-switch* por feature flag;
- Publicar **relatório mensal** por domínio com KPIs.

Métricas de sucesso (negócio + engenharia)

- **Qualidade:**
 - $\Delta\%$ redução de alucinação **confirmada por auditoria humana**;
 - Taxa de revisões automáticas aceitas ($\geq 70\%$).
- **Produtividade:**

- Redução de tempo de retrabalho/manual review;
- Aumento de *first-pass accept rate* em Q&A/FAQ.
- **Confiabilidade:**
 - p95 latência dentro do SLO;
 - erro operacional $< 0,5\%$;
 - incidentes P0 relacionados a *false block* = 0.
- **Adoção:**
 - % de rotas protegidas;
 - satisfação de stakeholders (NPS interno) sobre explicabilidade e *refs*.

Experimentos e ablações recomendadas

- **Ablation** por defesa: remover 1 módulo e medir Δ nos KPIs (identifica contribuição real).
- **Top-k retrieval:** k=4 vs 8 vs 12 (qual o ponto ótimo custo/ganho).
- **Cross-encoder:** MiniLM vs BERT-base (trade-off de p95).
- **Threshold sweep:** *tau_claim*, *coverage* — otimizar F1 entre *allow* e *review*.
- **Idioma:** PT vs EN — checar variação de precisão.

Roadmap (pós-MVP)

- **Short-term** (4–8 semanas):
 - Pesos por domínio (finance/health/general) para composição δ/ω ;
 - Regras numéricas/temporais mais robustas (detecção de conflitos de valores/datas);
 - Cache semântico por *claim* e por *blob* ($\downarrow$ custo/latência).
- **Mid-term** (8–16 semanas):
 - Classificador leve para “afirmação sem suporte” (substitui heurística em Density + Self-Evidence);
 - Integração com **observability** (traços distribuídos, drill-down por *claim* $\rightarrow$ evidência $\rightarrow$ doc);
 - Painel de **A/B** com alertas (ex.: aumento súbito de *unsupported_rate*).
- **Long-term:**
 - Ajuste adaptativo de thresholds por tenant (com feedback humano leve);
 - Integração *pre-generation* (reranking de context antes da resposta);
 - Suporte a *cross-provider* (afinando sinais para LLMs diferentes).

Riscos e mitigação

- **Contexto incompleto** → tendência a *revise/block*:
Mitigar com boa cobertura do RAG, mensagens claras ao usuário e *fallback* de pedir escopo/fonte.
 - **Falso positivo em estilos criativos:**
Ativar *mode=creative/instruction* que reduz peso de Confidence/Density.
 - **Custo/latência do cross-encoder:**
Tunar k e paralelizar; cache por claim normalizado; downgrade automático para lexical-only sob pressão.
 - **Múltiplas fontes com números distintos:**
Critérios de autoridade/frescor; mostrar *refs* múltiplas; não “fundir números” na revisão automática.
 - **Idiomas mistos:**
Embeddings multilíngues + listas de padrões PT/EN; normalização de datas/moedas.
-